



[Главная](#) >> [Команды](#) >> Команда grep в Linux

Команда grep в Linux

Обновлено: 10 апреля, 2023 **Опубликовано:** 22 августа, 2015 от [admin](#) , 18 комменариев, время чтения: 13 минут

Иногда может понадобится найти файл, в котором содержится определённая строка или найти строку в файле, где есть нужное слово. В Linux для этого существует несколько утилит, одна из самых используемых это grep. С её помощью можно искать не только строки в файлах, но и фильтровать вывод команд, и много чего ещё.

В этой инструкции мы рассмотрим что такое команда grep Linux, подробно разберём синтаксис и возможные опции grep, а также приведём несколько примеров работы с этой утилитой.

Содержание статьи:

- [Что такое grep?](#)
- [Синтаксис grep](#)
- [Опции](#)
- [Примеры использования grep](#)
 - [1. Поиск текста в файле](#)
 - [2. Фильтрация вывода команды](#)
 - [3. Базовые регулярные выражения](#)
 - [4. Расширенные регулярные выражения](#)
 - [5. Вывод контекста](#)



- [6. Рекурсивный поиск в grep](#)
- [7. Выбор файлов для поиска](#)
- [8. Поиск слов в grep](#)
- [9. Количество строк](#)
- [10. Инвертированный поиск](#)
- [11. Вывод имен файлов](#)
- [12. Цветной вывод](#)
- [Выводы](#)

Что такое grep?

Название команды **grep** расшифровывается как "search globally for lines matching the regular expression, and print them". Это одна из самых востребованных команд в терминале Linux, которая входит в состав проекта GNU. До того как появился проект GNU, существовала утилита предшественник grep, тем же названием, которая была разработана в 1973 году Кеном Томпсоном для поиска файлов по содержимому в Unix. А потом уже была разработана свободная утилита с той же функциональностью в рамках GNU.

Греп дает очень много возможностей для фильтрации текста. Вы можете выбирать нужные строки из текстовых файлов, отфильтровать вывод команд, и даже искать файлы в файловой системе, которые содержат определённые строки. Утилита очень популярна, потому что она уже предустановлена почти во всех дистрибутивах.

Синтаксис grep

Синтаксис команды выглядит следующим образом:

```
$ grep [опции] шаблон [/путь/к/файлу/или/папке...]
```

Или:

```
$ команда | grep [опции] шаблон
```

Здесь:

- **Опции** – это дополнительные параметры, с помощью которых указываются различные настройки поиска и вывода, например количество строк или режим инверсии.
- **Шаблон** – это любая строка или регулярное выражение, по которому будет выполняться поиск.
- **Имя файла или папки** – это то место, где будет выполняться поиск. Как вы увидите дальше, **grep** позволяет искать в нескольких файлах и даже в каталогах, используя рекурсивный режим.

Возможность фильтровать стандартный вывод пригодится, например, когда нужно выбрать только ошибки из логов или отфильтровать только необходимую информацию из вывода какой-либо другой утилиты.

Опции

Давайте рассмотрим самые основные опции утилиты, которые помогут более эффективно выполнять поиск текста в файлах `grep`:

- **-E, --extended-regexp** – включить расширенный режим регулярных выражений (ERE);
- **-F, --fixed-strings** – рассматривать шаблон поиска как обычную строку, а не регулярное выражение;
- **-G, --basic-regexp** – интерпретировать шаблон поиска как базовое регулярное выражение (BRE);
- **-P, --perl-regexp** – рассматривать шаблон поиска как регулярное выражение Perl;
- **-e, --regexp** – альтернативный способ указать шаблон поиска, опцию можно использовать несколько раз, что позволяет указать несколько шаблонов для поиска файлов, содержащих один из них;
- **-f, --file** – читать шаблон поиска из файла;
- **-i, --ignore-case** – не учитывать регистр символов;
- **-v, --invert-match** – вывести только те строки, в которых шаблон поиска не найден;
- **-w, --word-regexp** – искать шаблон как слово, отделенное пробелами или другими знаками препинания;
- **-x, --line-regexp** – искать шаблон как целую строку, от начала и до символа перевода строки;
- **-c** – вывести количество найденных строк;
- **--color** – включить цветной режим, доступные значения: **never**, **always** и **auto**;
- **-L, --files-without-match** – выводить только имена файлов, будут выведены все файлы в которых выполняется поиск;
- **-l, --files-with-match** – аналогично предыдущему, но будут выведены только файлы, в которых есть хотя бы одно вхождение;
- **-m, --max-count** – остановить поиск после того как будет найдено указанное количество строк;
- **-o, --only-matching** – отображать только совпавшую часть, вместо отображения всей строки;
- **-h, --no-filename** – не выводить имя файла;
- **-q, --quiet** – не выводить ничего;
- **-s, --no-messages** – не выводить ошибки чтения файлов;
- **-A, --after-content** – показать вхождение и n строк после него;
- **-B, --before-content** – показать вхождение и n строк после него;
- **-C** – показать n строк до и после вхождения;
- **-a, --text** – обрабатывать двоичные файлы как текст;

- **--exclude** – пропустить файлы имена которых соответствуют регулярному выражению;
- **--exclude-dir** – пропустить все файлы в указанной директории;
- **-I** – пропускать двоичные файлы;
- **--include** – искать только в файлах, имена которых соответствуют регулярному выражению;
- **-r** – рекурсивный поиск по всем подпапкам;
- **-R** – рекурсивный поиск включая ссылки;

Все самые основные опции рассмотрели, теперь давайте перейдём к примерам работы команды `grep` Linux.

Примеры использования `grep`

Давайте перейдём к практике. Сначала рассмотрим несколько основных примеров поиска внутри файлов Linux с помощью `grep`.

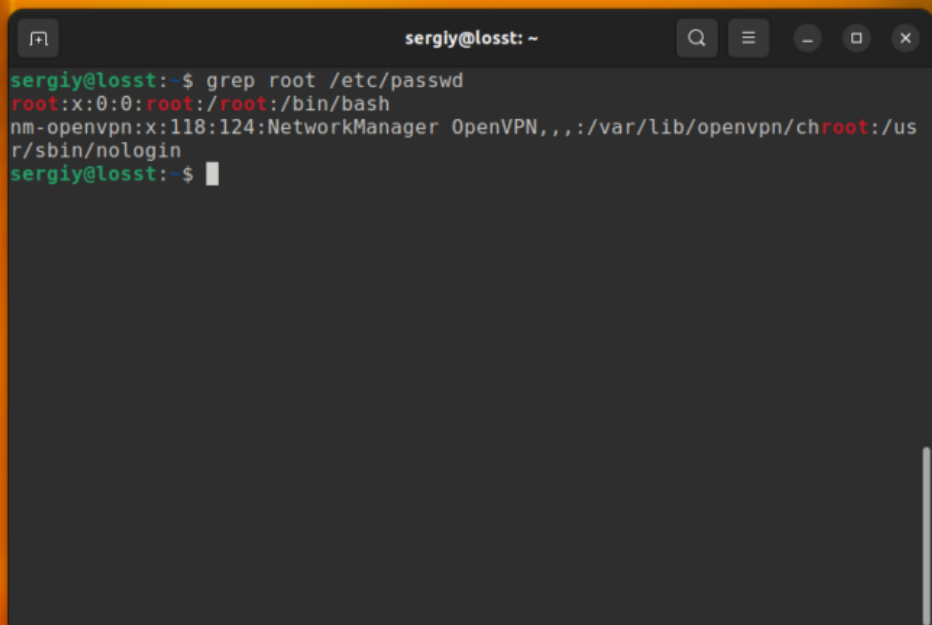
1. Поиск текста в файле

В первом примере мы будем искать информацию о пользователе **root** в файле со списком пользователей Linux **/etc/passwd**. Для этого выполните следующую команду:

```
$ grep root /etc/passwd
```

В результате вы получите что-то вроде этого:

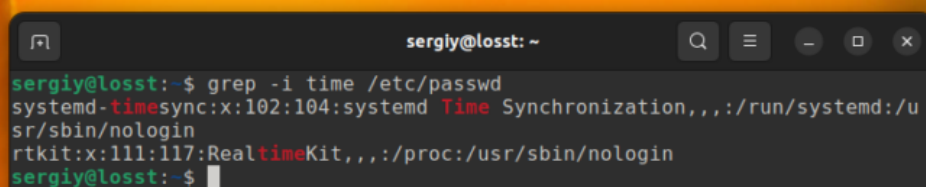




```
sergiy@losst: ~  
sergiy@losst:~$ grep root /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin  
sergiy@losst:~$
```

С помощью опции **-i** можно указать, что регистр символов учитывать не нужно. Например, давайте найдём все строки содержащие вхождение слова `time` в том же файле:

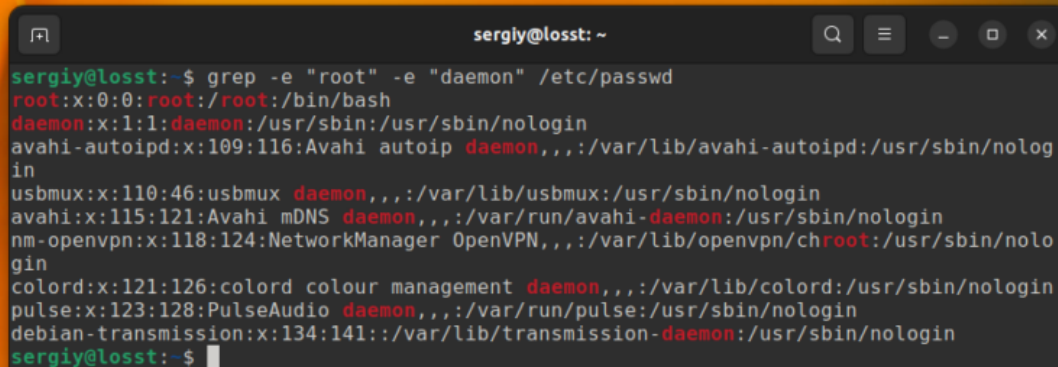
```
$ grep -i "time" /etc/passwd
```



```
sergiy@losst: ~  
sergiy@losst:~$ grep -i time /etc/passwd  
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin  
rtkit:x:111:117:RealtimeKit,,,:/proc:/usr/sbin/nologin  
sergiy@losst:~$
```

В этом случае **Time**, **time**, **TIME** и другие вариации слова будут считаться эквивалентными. Ещё, вы можете указать несколько условий для поиска, используя опцию **-e**. Например:

```
$ grep -e "root" -e "daemon" /etc/passwd
```

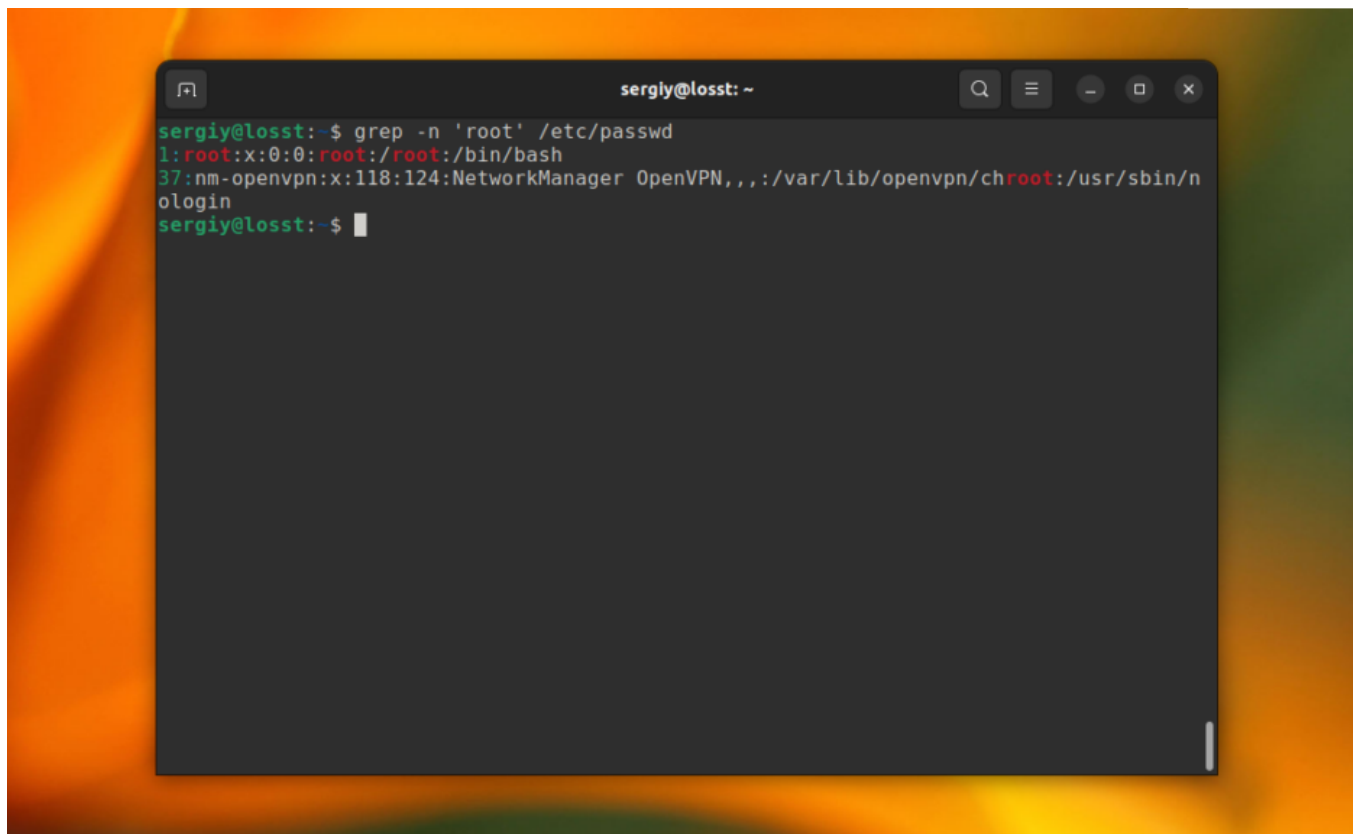


```
sergiy@losst: ~  
sergiy@losst:~$ grep -e "root" -e "daemon" /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin  
avahi-autoipd:x:109:116:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin  
usbmux:x:110:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin  
avahi:x:115:121:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin  
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin  
colord:x:121:126:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin  
pulse:x:123:128:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin  
debian-transmission:x:134:141:./var/lib/transmission-daemon:/usr/sbin/nologin  
sergiy@losst:~$
```



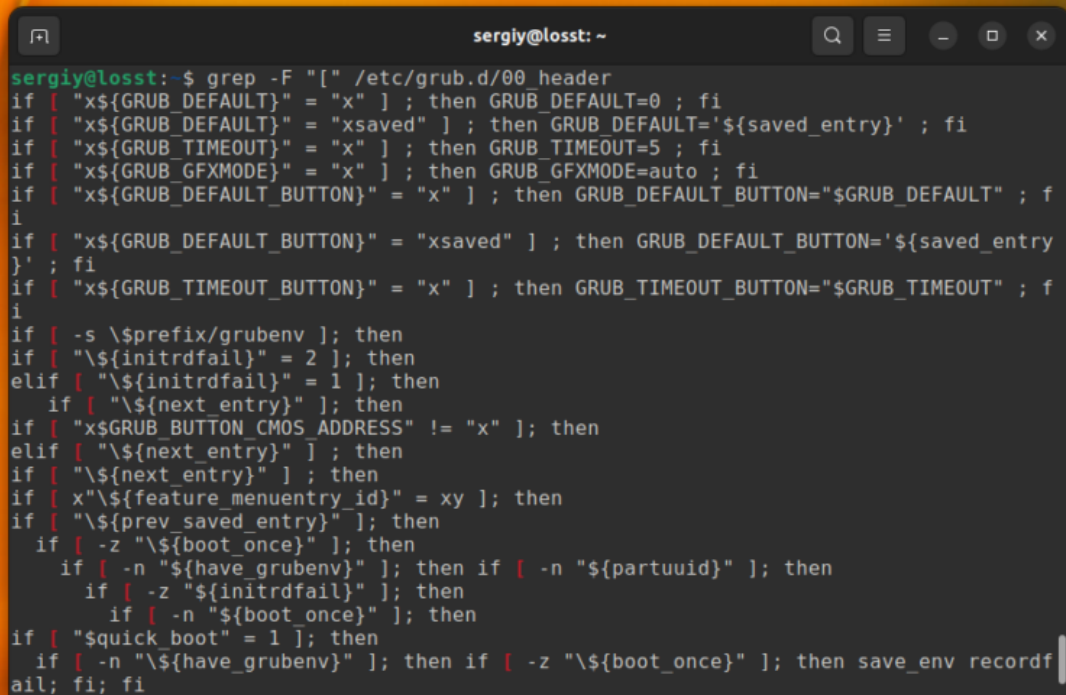
С помощью опции **-n** можно выводить номер строки, в которой найдено вхождение, например:

```
$ grep -n 'root' /etc/passwd
```

A terminal window titled 'sergiy@losst: ~' with standard window controls. The command 'grep -n 'root' /etc/passwd' has been executed. The output shows three lines from the /etc/passwd file: '1:root:x:0:0:root:/root:/bin/bash', '37:nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin', and the prompt 'sergiy@losst:~\$'.

Это всё хорошо работает пока ваш поисковый запрос не содержит специальных символов. Например, если вы попытаетесь найти все строки, которые содержат символ "[" в файле /etc/grub/00_header, то получите ошибку, что это регулярное выражение не верно. Для того чтобы этого избежать, нужно явно указать, что вы хотите искать строку с помощью опции **-F**:

```
$ grep -F "[" /etc/grub.d/00_header
```

A terminal window titled 'sergiy@losst: ~' showing the output of the command 'grep -F "[" /etc/grub.d/00_header'. The output is a series of conditional statements from the GRUB header file, such as 'if ["x\${GRUB_DEFAULT}" = "x"] ; then GRUB_DEFAULT=0 ; fi' and 'if ["x\${GRUB_TIMEOUT}" = "x"] ; then GRUB_TIMEOUT=5 ; fi'.

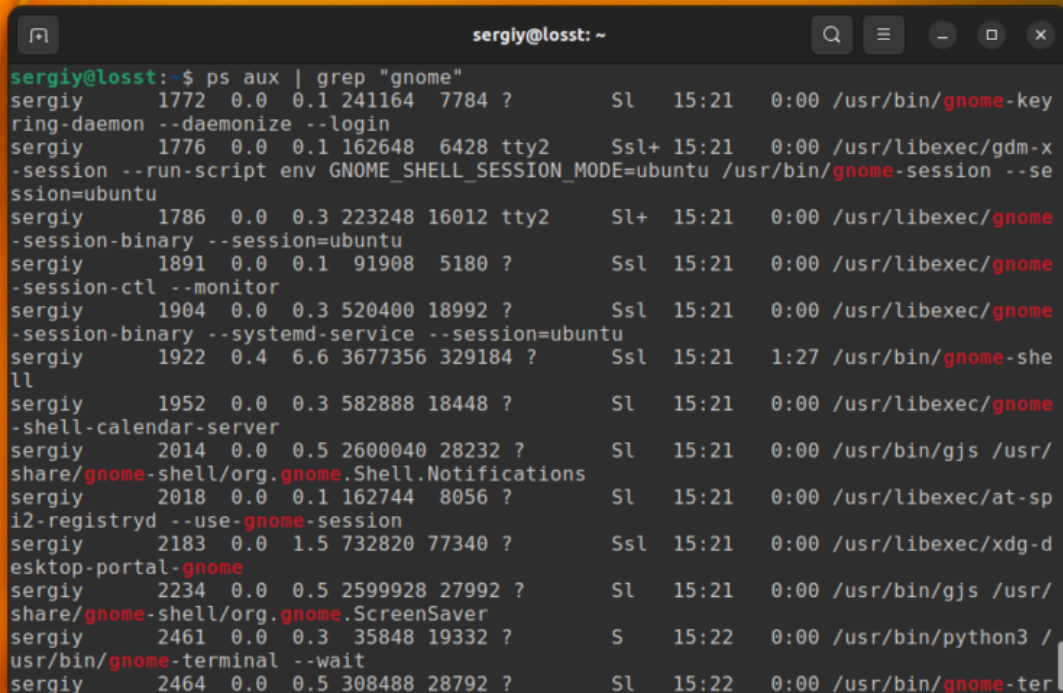
```
sergiy@losst:~$ grep -F "[" /etc/grub.d/00_header
if [ "x${GRUB_DEFAULT}" = "x" ] ; then GRUB_DEFAULT=0 ; fi
if [ "x${GRUB_DEFAULT}" = "xsaved" ] ; then GRUB_DEFAULT='${saved_entry}' ; fi
if [ "x${GRUB_TIMEOUT}" = "x" ] ; then GRUB_TIMEOUT=5 ; fi
if [ "x${GRUB_GFXMODE}" = "x" ] ; then GRUB_GFXMODE=auto ; fi
if [ "x${GRUB_DEFAULT_BUTTON}" = "x" ] ; then GRUB_DEFAULT_BUTTON="${GRUB_DEFAULT}" ; fi
if [ "x${GRUB_DEFAULT_BUTTON}" = "xsaved" ] ; then GRUB_DEFAULT_BUTTON='${saved_entry}' ; fi
if [ "x${GRUB_TIMEOUT_BUTTON}" = "x" ] ; then GRUB_TIMEOUT_BUTTON="${GRUB_TIMEOUT}" ; fi
if [ -s \${prefix}/grubenv ] ; then
if [ "\${initrdfail}" = 2 ] ; then
elif [ "\${initrdfail}" = 1 ] ; then
    if [ "\${next_entry}" ] ; then
if [ "x${GRUB_BUTTON_CMOS_ADDRESS}" != "x" ] ; then
elif [ "\${next_entry}" ] ; then
if [ "\${next_entry}" ] ; then
if [ x"${feature_menuentry_id}" = xy ] ; then
if [ "\${prev_saved_entry}" ] ; then
    if [ -z "\${boot_once}" ] ; then
        if [ -n "\${have_grubenv}" ] ; then if [ -n "${partuuid}" ] ; then
            if [ -z "\${initrdfail}" ] ; then
                if [ -n "\${boot_once}" ] ; then
if [ "$quick boot" = 1 ] ; then
    if [ -n "\${have_grubenv}" ] ; then if [ -z "\${boot_once}" ] ; then save_env recordfail; fi; fi
```

Теперь вы знаете как выполняется поиск текста файлах `grep`.

2. Фильтрация вывода команды

Для того чтобы отфильтровать вывод другой команды с помощью `grep` достаточно перенаправить его используя оператор `|`. А файл для самого `grep` указывать не надо. Например, для того чтобы найти все процессы `gnome` можно использовать такую команду:

```
$ ps aux | grep "gnome"
```



```
sergiy@losst: ~  
sergiy@losst:~$ ps aux | grep "gnome"  
sergiy      1772  0.0  0.1 241164  7784 ?        Ssl  15:21   0:00 /usr/bin/gnome-key  
ring-daemon --daemonize --login  
sergiy      1776  0.0  0.1 162648  6428 tty2      Ssl+ 15:21   0:00 /usr/libexec/gdm-x  
-session --run-script env GNOME_SHELL_SESSION_MODE=ubuntu /usr/bin/gnome-session --se  
ssion=ubuntu  
sergiy      1786  0.0  0.3 223248 16012 tty2      Ssl+ 15:21   0:00 /usr/libexec/gnome  
-session-binary --session=ubuntu  
sergiy      1891  0.0  0.1  91908  5180 ?        Ssl  15:21   0:00 /usr/libexec/gnome  
-session-ctl --monitor  
sergiy      1904  0.0  0.3 520400 18992 ?        Ssl  15:21   0:00 /usr/libexec/gnome  
-session-binary --systemd-service --session=ubuntu  
sergiy      1922  0.4  6.6 3677356 329184 ?       Ssl  15:21   1:27 /usr/bin/gnome-she  
ll  
sergiy      1952  0.0  0.3 582888 18448 ?        Ssl  15:21   0:00 /usr/libexec/gnome  
-shell-calendar-server  
sergiy      2014  0.0  0.5 2600040 28232 ?        Ssl  15:21   0:00 /usr/bin/gjs /usr/  
share/gnome-shell/org.gnome.Shell.Notifications  
sergiy      2018  0.0  0.1 162744  8056 ?        Ssl  15:21   0:00 /usr/libexec/at-sp  
i2-registryd --use-gnome-session  
sergiy      2183  0.0  1.5 732820  77340 ?       Ssl  15:21   0:00 /usr/libexec/xdg-d  
esktop-portal-gnome  
sergiy      2234  0.0  0.5 2599928 27992 ?        Ssl  15:21   0:00 /usr/bin/gjs /usr/  
share/gnome-shell/org.gnome.ScreenSaver  
sergiy      2461  0.0  0.3  35848 19332 ?        S    15:22   0:00 /usr/bin/python3 /  
usr/bin/gnome-terminal --wait  
sergiy      2464  0.0  0.5 308488 28792 ?        Ssl  15:22   0:00 /usr/bin/gnome-ter
```

В остальном всё работает аналогично.

3. Базовые регулярные выражения

Утилита `grep` поддерживает несколько видов регулярных выражений. Это базовые регулярные выражения (BRE), которые используются по умолчанию и расширенные (ERE). Базовые регулярные выражение поддерживает набор символов, позволяющих описать каждый определённый символ в строке. Это: `.`, `*`, `[]`, `[^]`, `^` и `$`. Например, вы можете найти строки, которые начинаются на букву `r`:

```
$ grep "^r" /etc/passwd
```



```

sergiy@losst: ~
sergiy@losst:~$ grep "^r" /etc/passwd
root:x:0:0:root:/root:/bin/bash
rtkit:x:111:117:RealtimeKit,,,:/proc:/usr/sbin/nologin
sergiy@losst:~$

```

Или же строки, которые содержат большие буквы:

```
$ grep "[A-Z]" /etc/passwd
```

```

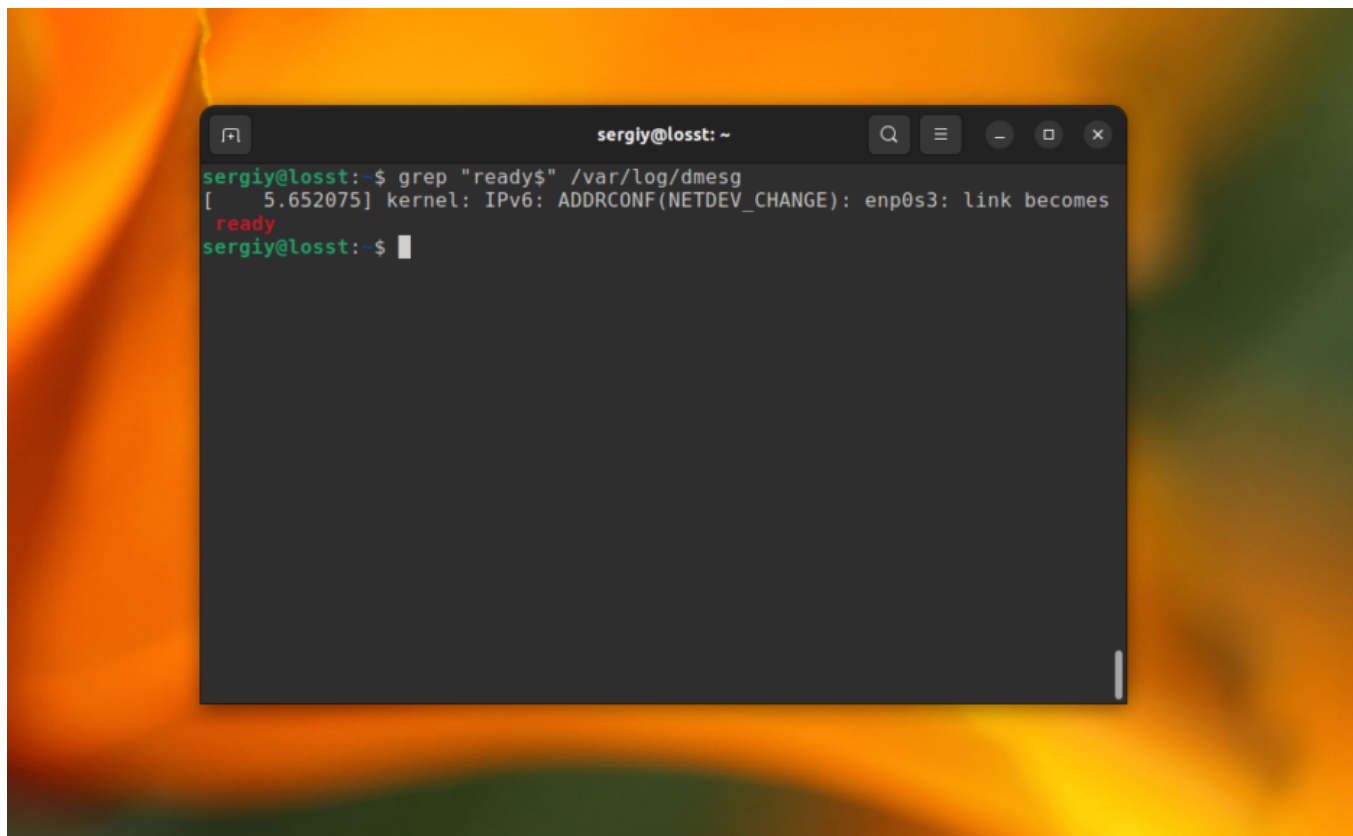
sergiy@losst: ~
sergiy@losst:~$ grep "[A-Z]" /etc/passwd
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
systemd-network:x:100:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:101:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
systemd-timesync:x:102:104:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
avahi-autoipd:x:109:116:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr/sbin/nologin
rtkit:x:111:117:RealtimeKit,,,:/proc:/usr/sbin/nologin
speech-dispatcher:x:114:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
avahi:x:115:121:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nologin
kernoops:x:116:65534:Kernel Oops Tracking Daemon,,,:/usr/sbin/nologin
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
hplip:x:119:7:HPLIP system user,,,:/run/hplip:/bin/false
pulse:x:123:128:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin

```

А так можно найти все строки, которые заканчиваются на ready в файле `/var/log/` :

Privacy

```
$ grep "ready$" /var/log/dmesg
```



Но используя базовый синтаксис вы не можете указать точное количество этих символов.

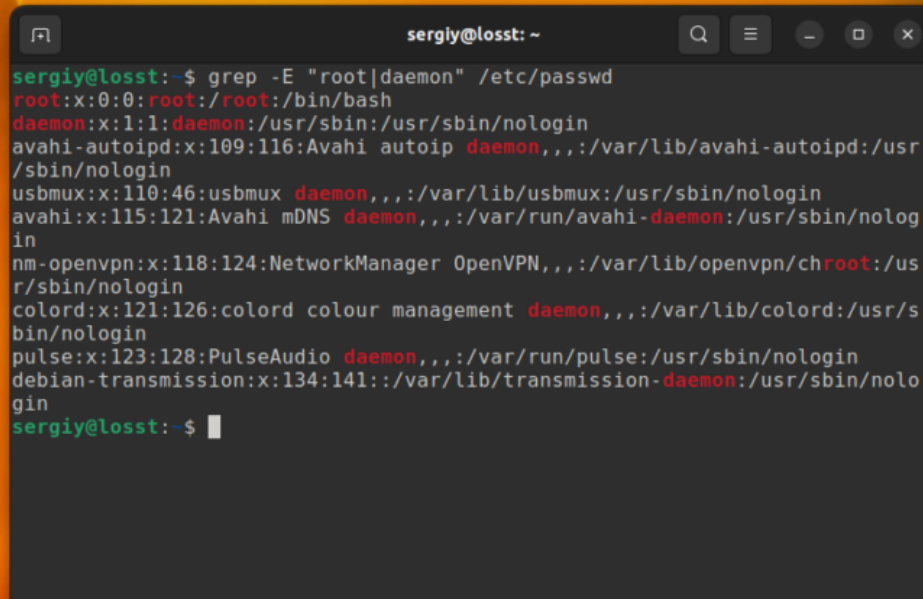
4. Расширенные регулярные выражения

В дополнение ко всем символам из базового синтаксиса, в расширенном синтаксисе поддерживаются также такие символы:

- **+** – одно или больше повторений предыдущего символа;
- **?** – ноль или одно повторение предыдущего символа;
- **{n,m}** – повторение предыдущего символа от n до m раз;
- **|** – позволяет объединять несколько паттернов.

Для активации расширенного синтаксиса нужно использовать опцию **-E**. Например, вместо использования опции **-e**, можно объединить несколько слов для поиска вот так:

```
$ grep -E "root|daemon" /etc/passwd
```

A terminal window titled 'sergiy@losst: ~' with search, menu, and window control icons in the title bar. The terminal shows the command 'grep -E "root|daemon" /etc/passwd' and its output, which lists system users like root, daemon, avahi-autoipd, usbmux, and others, with their respective IDs, names, and shell paths. The output is color-coded: usernames are green, IDs are red, and names are white. The prompt 'sergiy@losst:~\$' is at the bottom.

```
sergiy@losst:~$ grep -E "root|daemon" /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
avahi-autoipd:x:109:116:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/usr
/sbin/nologin
usbmux:x:110:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
avahi:x:115:121:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/usr/sbin/nolog
in
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/us
r/sbin/nologin
colord:x:121:126:colord colour management daemon,,,:/var/lib/colord:/usr/s
bin/nologin
pulse:x:123:128:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
debian-transmission:x:134:141:./var/lib/transmission-daemon:/usr/sbin/nolo
gin
sergiy@losst:~$
```

Вообще, регулярные выражения `grep` – это очень обширная тема, в этой статье я лишь показал несколько примеров. Как вы увидели, поиск текста в файлах `grep` становится ещё эффективнее. Но на полное объяснение этой темы нужна целая статья, поэтому пока пропустим её и пойдем дальше.

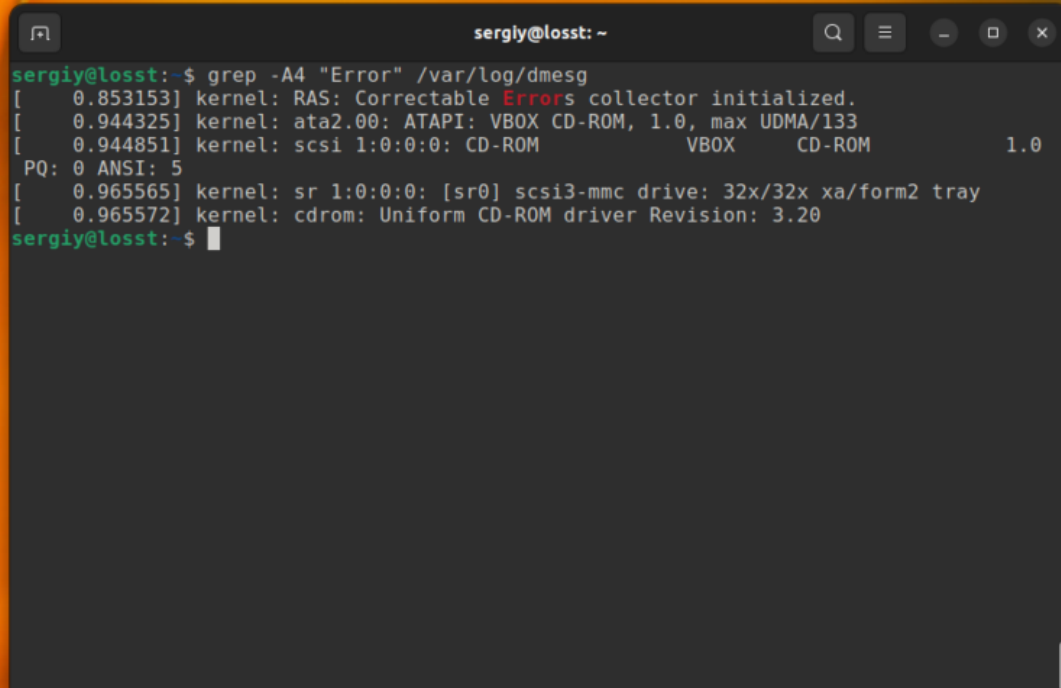
5. Вывод контекста

Иногда бывает очень полезно вывести не только саму строку со вхождением, но и строки до и после неё. Например, мы хотим выбрать все ошибки из лог-файла, но знаем, что в следующей строчке после ошибки может содержаться полезная информация, тогда с помощью `grep` отобразим несколько строк. Ошибки будем искать в `/var/log/dmesg` по шаблону `"Error"`:

```
$ grep -A4 "Error" /var/log/dmesg
```

Выведет строку с вхождением и 4 строчки после неё:



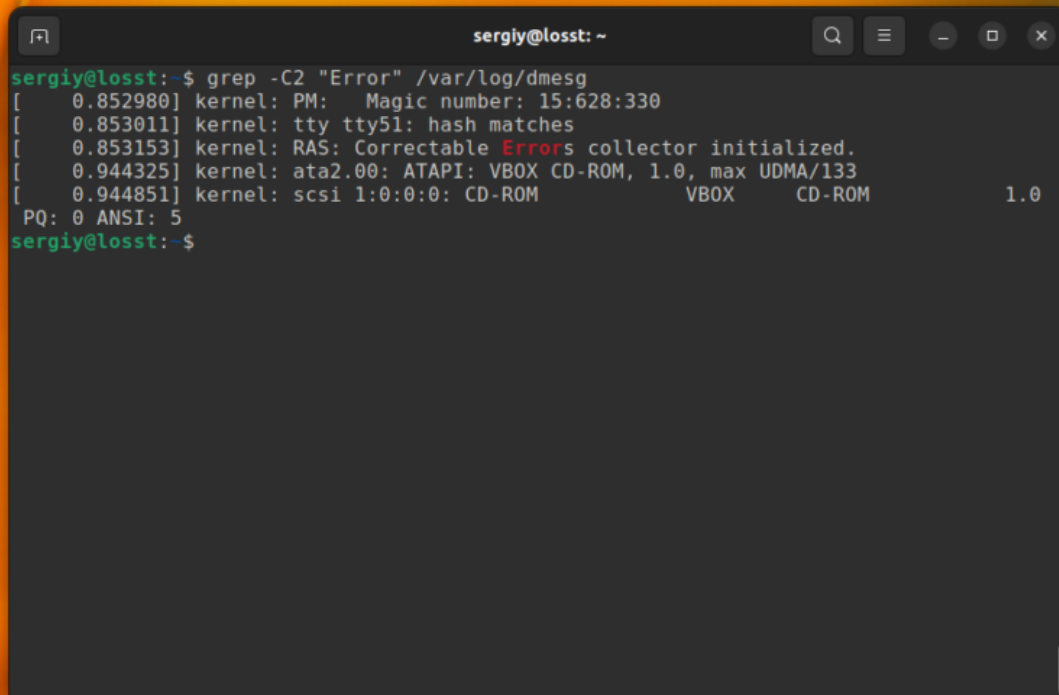
A screenshot of a terminal window titled 'sergiy@losst: ~'. The terminal shows the command 'grep -A4 "Error" /var/log/dmesg' and its output. The output consists of five lines of kernel log messages, with the word 'Errors' highlighted in red in the first line. The messages are: '[0.853153] kernel: RAS: Correctable Errors collector initialized.', '[0.944325] kernel: ata2.00: ATAPI: VBOX CD-ROM, 1.0, max UDMA/133', '[0.944851] kernel: scsi 1:0:0:0: CD-ROM VBOX CD-ROM 1.0', 'PQ: 0 ANSI: 5', and '[0.965565] kernel: sr 1:0:0:0: [sr0] scsi3-mmc drive: 32x/32x xa/form2 tray'. The prompt 'sergiy@losst:~\$' is visible at the bottom.

```
sergiy@losst:~$ grep -A4 "Error" /var/log/dmesg
[ 0.853153] kernel: RAS: Correctable Errors collector initialized.
[ 0.944325] kernel: ata2.00: ATAPI: VBOX CD-ROM, 1.0, max UDMA/133
[ 0.944851] kernel: scsi 1:0:0:0: CD-ROM VBOX CD-ROM 1.0
PQ: 0 ANSI: 5
[ 0.965565] kernel: sr 1:0:0:0: [sr0] scsi3-mmc drive: 32x/32x xa/form2 tray
[ 0.965572] kernel: cdrom: Uniform CD-ROM driver Revision: 3.20
sergiy@losst:~$
```

```
$ grep -B4 "Error" /var/log/dmesg
```

Эта команда выведет строку со вхождением и 4 строчки до неё. А следующая выведет по две строки с верха и снизу от вхождения.

```
$ grep -C2 "Error" /var/log/dmesg
```

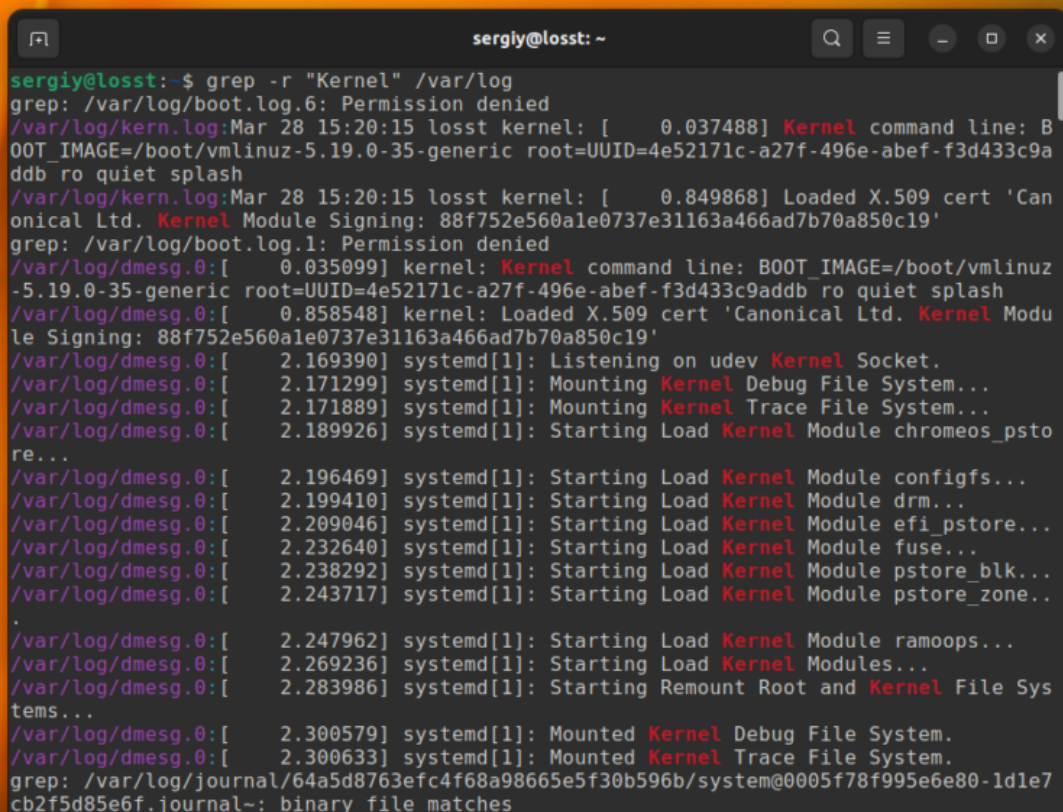
A terminal window titled 'sergiy@losst: ~' with search, menu, and window control icons in the title bar. The terminal shows the command 'grep -C2 "Error" /var/log/dmesg' and its output. The output consists of several lines of kernel messages, with the word 'Errors' in red in the third line. The terminal text is as follows:

```
sergiy@losst:~$ grep -C2 "Error" /var/log/dmesg
[ 0.852980] kernel: PM: Magic number: 15:628:330
[ 0.853011] kernel: tty tty51: hash matches
[ 0.853153] kernel: RAS: Correctable Errors collector initialized.
[ 0.944325] kernel: ata2.00: ATAPI: VBOX CD-ROM, 1.0, max UDMA/133
[ 0.944851] kernel: scsi 1:0:0:0: CD-ROM VBOX CD-ROM 1.0
PQ: 0 ANSI: 5
sergiy@losst:~$
```

6. Рекурсивный поиск в grep

До этого мы рассматривали поиск в определённом файле или выводе команд. Но `grep` также может выполнить поиск текста в нескольких файлах, размещённых в одном каталоге или подкаталогах. Для этого нужно использовать опцию `-r`. Например, давайте найдём все файлы, которые содержат строку **kernel** в папке **/var/log**:

```
$ grep -r "kernel" /var/log
```



```
sergij@losst:~$ grep -r "Kernel" /var/log
grep: /var/log/boot.log.6: Permission denied
/var/log/kern.log:Mar 28 15:20:15 losst kernel: [ 0.037488] Kernel command line: BOOT_IMAGE=/boot/vmlinuz-5.19.0-35-generic root=UUID=4e52171c-a27f-496e-abef-f3d433c9a
ddb ro quiet splash
/var/log/kern.log:Mar 28 15:20:15 losst kernel: [ 0.849868] Loaded X.509 cert 'Can
onical Ltd. Kernel Module Signing: 88f752e560ale0737e31163a466ad7b70a850c19'
grep: /var/log/boot.log.1: Permission denied
/var/log/dmesg.0:[ 0.035099] kernel: Kernel command line: BOOT_IMAGE=/boot/vmlinuz
-5.19.0-35-generic root=UUID=4e52171c-a27f-496e-abef-f3d433c9adbb ro quiet splash
/var/log/dmesg.0:[ 0.858548] kernel: Loaded X.509 cert 'Canonical Ltd. Kernel Modu
le Signing: 88f752e560ale0737e31163a466ad7b70a850c19'
/var/log/dmesg.0:[ 2.169390] systemd[1]: Listening on udev Kernel Socket.
/var/log/dmesg.0:[ 2.171299] systemd[1]: Mounting Kernel Debug File System...
/var/log/dmesg.0:[ 2.171889] systemd[1]: Mounting Kernel Trace File System...
/var/log/dmesg.0:[ 2.189926] systemd[1]: Starting Load Kernel Module chromeos_psto
re...
/var/log/dmesg.0:[ 2.196469] systemd[1]: Starting Load Kernel Module configs...
/var/log/dmesg.0:[ 2.199410] systemd[1]: Starting Load Kernel Module drm...
/var/log/dmesg.0:[ 2.209046] systemd[1]: Starting Load Kernel Module efi_pstore...
/var/log/dmesg.0:[ 2.232640] systemd[1]: Starting Load Kernel Module fuse...
/var/log/dmesg.0:[ 2.238292] systemd[1]: Starting Load Kernel Module pstore_blk...
/var/log/dmesg.0:[ 2.243717] systemd[1]: Starting Load Kernel Module pstore_zone...
/var/log/dmesg.0:[ 2.247962] systemd[1]: Starting Load Kernel Module ramoops...
/var/log/dmesg.0:[ 2.269236] systemd[1]: Starting Load Kernel Modules...
/var/log/dmesg.0:[ 2.283986] systemd[1]: Starting Remount Root and Kernel File Sys
tems...
/var/log/dmesg.0:[ 2.300579] systemd[1]: Mounted Kernel Debug File System.
/var/log/dmesg.0:[ 2.300633] systemd[1]: Mounted Kernel Trace File System.
grep: /var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@0005f78f995e6e80-1dle7
cb2f5d85e6f.journal~: binary file matches
```

Папка с вашими файлами может содержать двоичные файлы, в которых поиск выполнять обычно не надо. Для того чтобы их пропускать используйте опцию **-I**:

```
$ grep -rI "Kernel" /var/log
```

Некоторые файлы доступны только суперпользователю и для того чтобы выполнять по ним поиск вам нужно запускать grep с помощью sudo. Или же вы можете просто скрыть сообщения об ошибках чтения и пропускать такие файлы с помощью опции **-s**:

```
$ grep -rIs "Kernel" /var/log
```

7. Выбор файлов для поиска

С помощью опций **--include** и **--exclude** вы можете фильтровать файлы, которые будут принимать участие в поиске. Например, для того чтобы выполнить поиск только по файлам с расширением **.log** в папке **/var/log** используйте такую команду:

```
$ grep -r --include="*.log" "Kernel" /var/log
```

```
sergiy@losst: ~  
sergiy@losst:~$ grep -r --include *.log "Kernel" /var/log  
/var/log/kern.log:Mar 28 15:20:15 losst kernel: [ 0.037488] Kernel command line: BOOT_IMAGE=/boot/vmlinuz-5.19.0-35-generic root=UUID=4e52171c-a27f-496e-abef-f3d433c9a  
ddb ro quiet splash  
/var/log/kern.log:Mar 28 15:20:15 losst kernel: [ 0.849868] Loaded X.509 cert 'Can  
onical Ltd. Kernel Module Signing: 88f752e560ale0737e31163a466ad7b70a850c19'  
grep: /var/log/speech-dispatcher: Permission denied  
/var/log/dist-upgrade/main.log:2022-04-24 20:48:49,147 DEBUG Kernel uname: '5.13.0-40  
-generic'  
/var/log/dist-upgrade/main.log:2022-04-24 21:07:34,660 DEBUG Kernel uname: '5.13.0-40  
-generic'  
grep: /var/log/boot.log: Permission denied  
grep: /var/log/kibana: Permission denied  
grep: /var/log/php8.1-fpm.log: Permission denied  
grep: /var/log/sss: Permission denied  
grep: /var/log/installer/casper.log: Permission denied  
/var/log/auth.log:Mar 28 21:21:43 losst sudo: sergiy : TTY=pts/0 ; PWD=/home/sergiy  
; USER=root ; COMMAND=/usr/bin/grep -lr Kernel /var/log/  
/var/log/auth.log:Mar 28 21:21:44 losst sudo: sergiy : TTY=pts/0 ; PWD=/home/sergiy  
; USER=root ; COMMAND=/usr/bin/grep -lr Kernel /var/log/  
/var/log/auth.log:Mar 28 21:22:12 losst sudo: sergiy : TTY=pts/0 ; PWD=/home/sergiy  
; USER=root ; COMMAND=/usr/bin/grep -lr Kernel /var/log/  
grep: /var/log/private: Permission denied  
grep: /var/log/gdm3: Permission denied  
sergiy@losst:~$
```

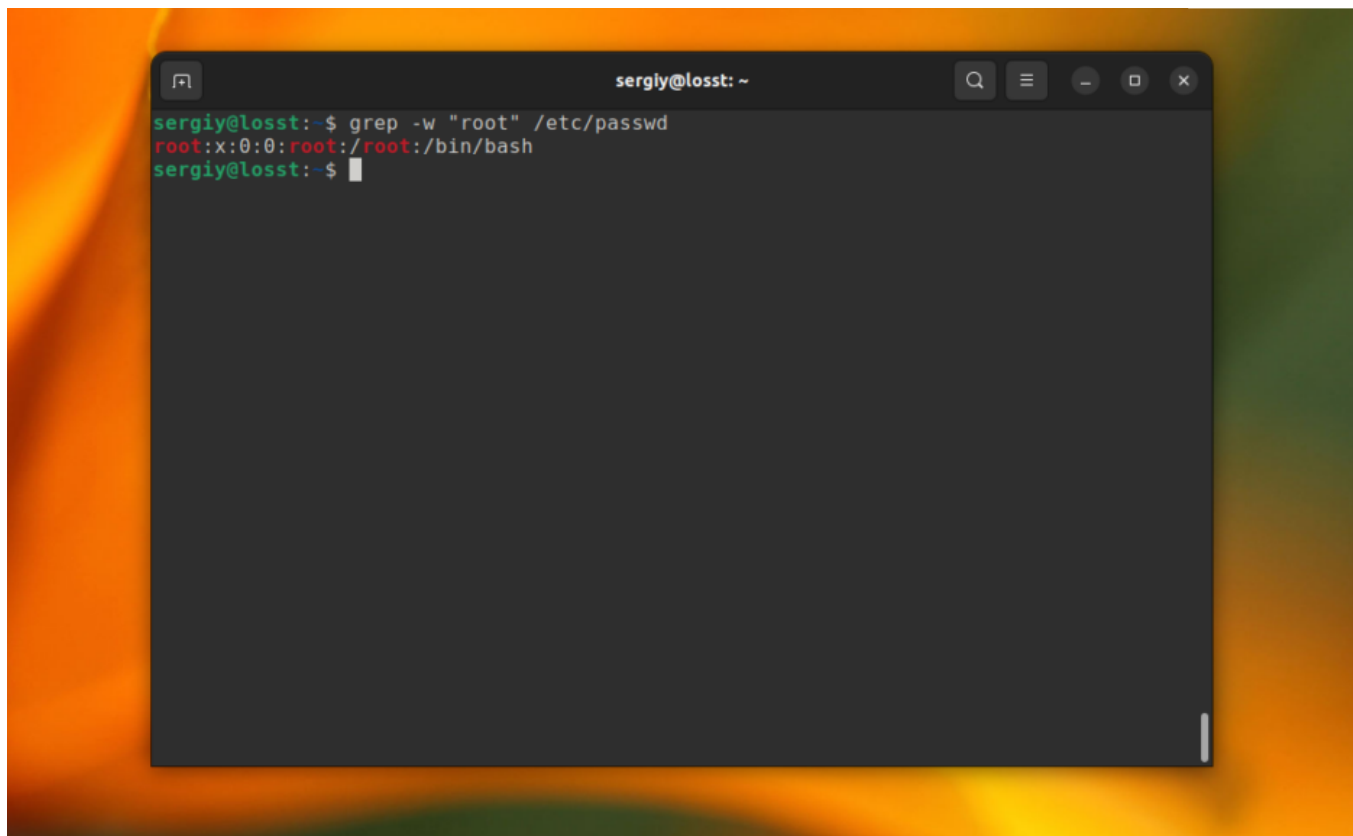
А для того чтобы исключить все файлы с расширением **.journal** надо использовать опцию **--exclude**:

```
$ grep -r --exclude="*.journal" "Kernel" /var/log
```

8. Поиск слов в grep

Когда вы ищете строку **abc**, **grep** будет выводить также **kabc**, **abc123**, **aafrabc32** и тому подобные комбинации. Вы можете заставить утилиту искать по содержимому файлов в Linux строки, которые включают только искомые слова полностью с помощью опции **-w**. Например:

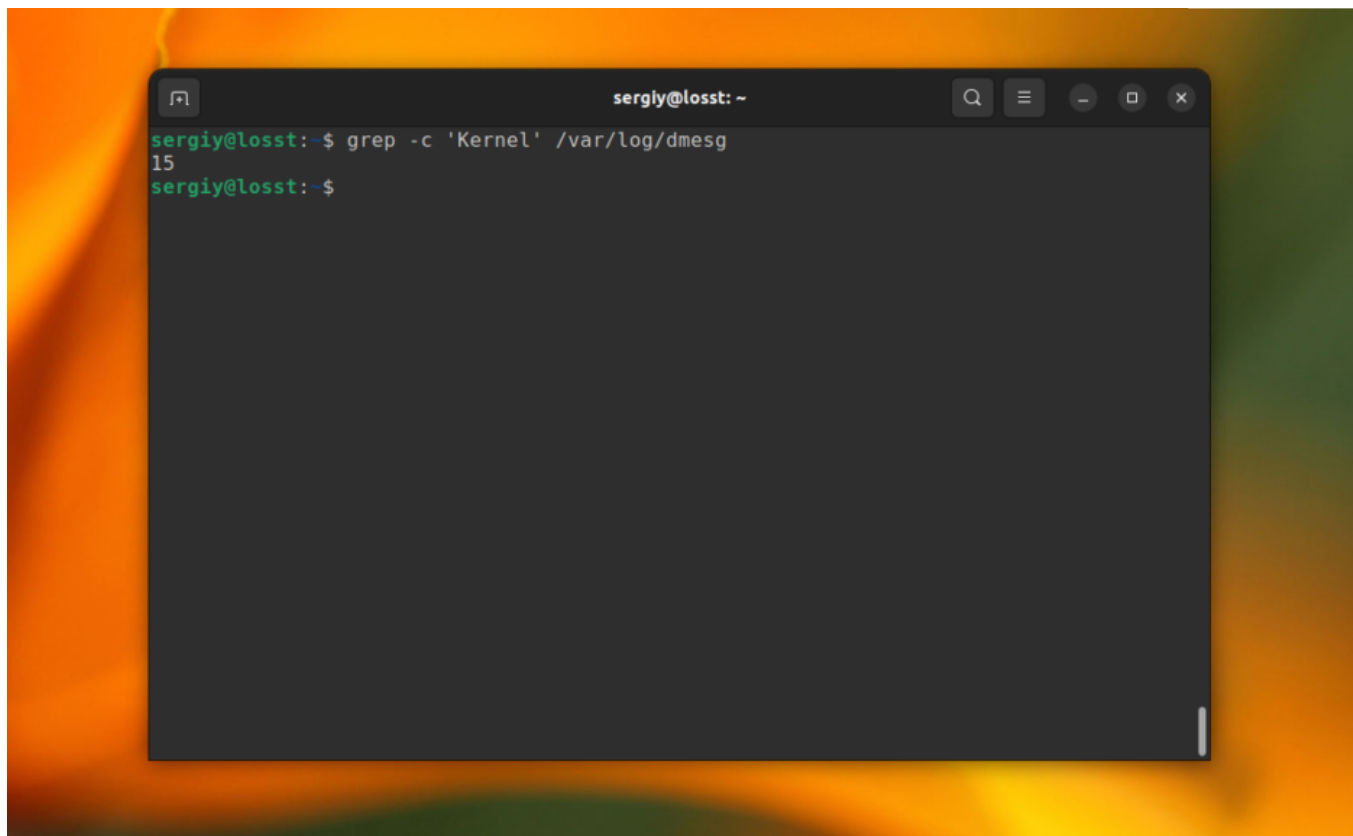
```
$ grep -w "root" /etc/passwd
```

A terminal window titled 'sergly@losst: ~' with standard window controls. The command 'grep -w "root" /etc/passwd' is entered. The output shows the root user entry: 'root:x:0:0:root:/root:/bin/bash'. The prompt returns to 'sergly@losst: \$'.

9. Количество строк

Утилита **grep** может сообщить, сколько строк с определенным текстом было найдено файле. Для этого используется опция **-c** (счетчик). Например:

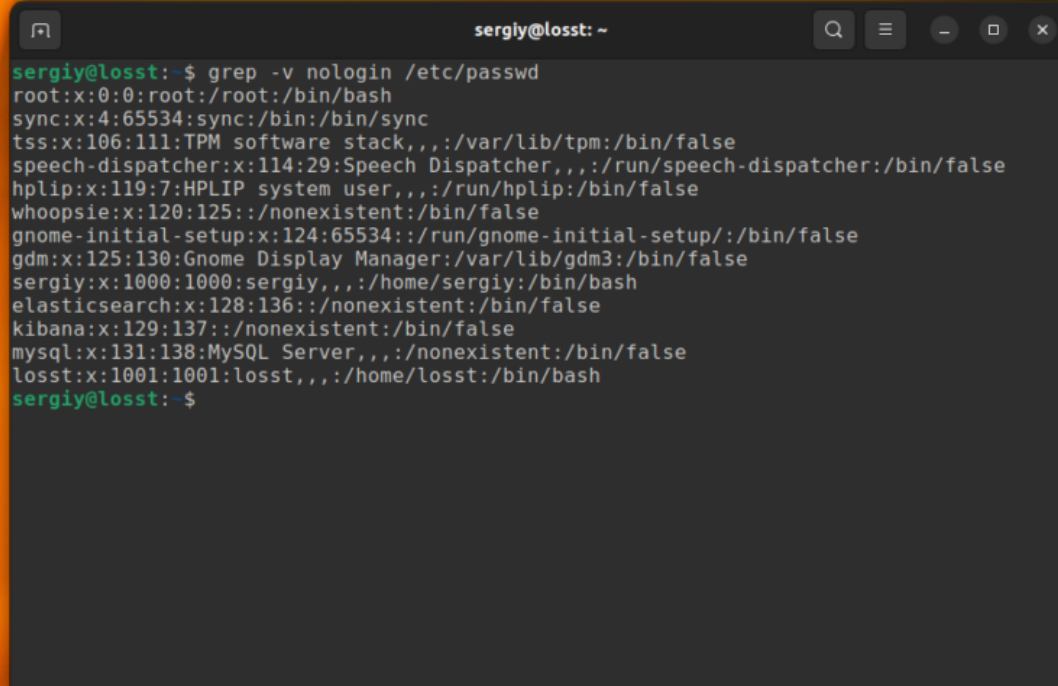
```
$ grep -c 'Kernel' /var/log/dmesg
```



10. Инвертированный поиск

Команда **grep** Linux может быть использована для поиска строк, которые не содержат указанное слово. Например, так можно вывести только те строки, которые не содержат слово **nologin**:

```
$ grep -v nologin /etc/passwd
```

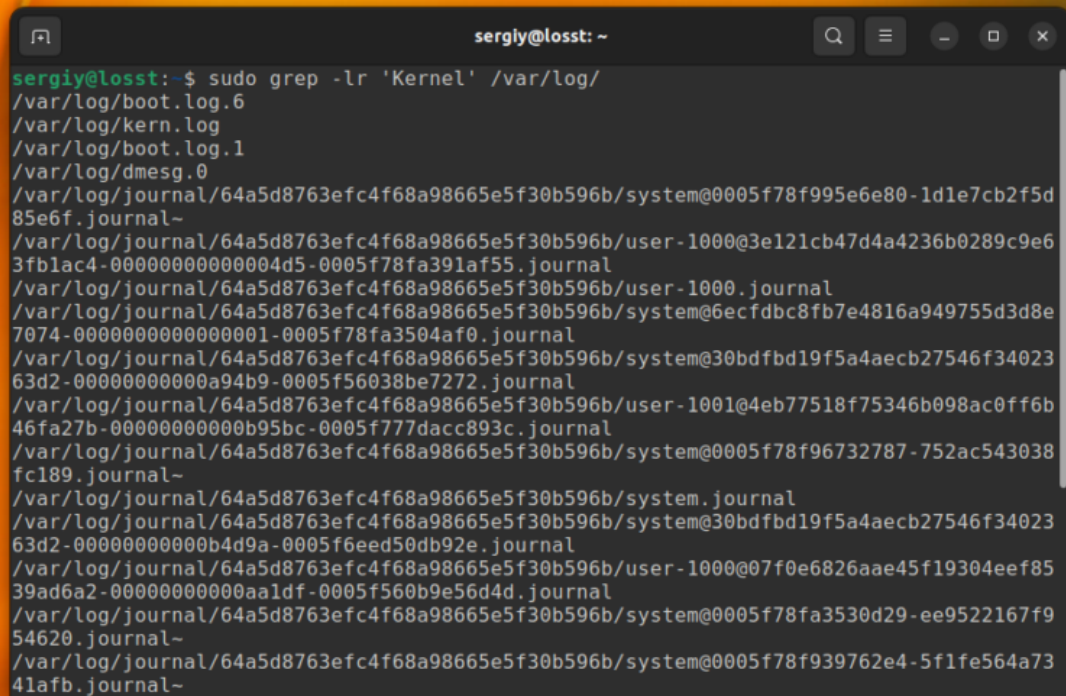
A terminal window titled 'sergiy@losst: ~' with standard window controls. It displays the output of the command 'grep -v nologin /etc/passwd'. The output lists system and user accounts, excluding those with a 'nologin' shell. The accounts listed are root, sync, tss, speech-dispatcher, hplip, whoopsie, gnome-initial-setup, gdm, sergiy, elasticsearch, kibana, mysql, and losst. Each entry shows the username, UID, GID, name, comment, home directory, and shell path.

```
sergiy@losst:~$ grep -v nologin /etc/passwd
root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
tss:x:106:111:TPM software stack,,,:/var/lib/tpm:/bin/false
speech-dispatcher:x:114:29:Speech Dispatcher,,,:/run/speech-dispatcher:/bin/false
hplip:x:119:7:HPLIP system user,,,:/run/hplip:/bin/false
whoopsie:x:120:125:./nonexistent:/bin/false
gnome-initial-setup:x:124:65534:./run/gnome-initial-setup:/bin/false
gdm:x:125:130:Gnome Display Manager:/var/lib/gdm3:/bin/false
sergiy:x:1000:1000:sergiy,,,:/home/sergiy:/bin/bash
elasticsearch:x:128:136:./nonexistent:/bin/false
kibana:x:129:137:./nonexistent:/bin/false
mysql:x:131:138:MySQL Server,,,:/nonexistent:/bin/false
losst:x:1001:1001:losst,,,:/home/losst:/bin/bash
sergiy@losst:~$
```

11. Вывод имен файлов

Вы можете указать **grep** выводить только имена файлов, в которых было хотя бы одно вхождение с помощью опции **-l**. Например, следующая команда выведет все имена файлов из каталога `/var/log`, при поиске по содержимому которых было обнаружено вхождение **Kernel**:

```
$ grep -lr 'Kernel' /var/log/
```


A terminal window titled 'sergly@losst: ~' with standard window controls. It shows the command 'sudo grep -lr 'Kernel' /var/log/' and its output, which lists various log files in the /var/log directory, including boot logs, kernel logs, dmesg, and several journal files with their full paths.

```
sergly@losst: ~  
sergly@losst:~$ sudo grep -lr 'Kernel' /var/log/  
/var/log/boot.log.6  
/var/log/kern.log  
/var/log/boot.log.1  
/var/log/dmesg.0  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@0005f78f995e6e80-1d1e7cb2f5d85e6f.journal~  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/user-1000@3e121cb47d4a4236b0289c9e63fblac4-00000000000004d5-0005f78fa391af55.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/user-1000.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@6ecfdb8fb7e4816a949755d3d8e7074-0000000000000001-0005f78fa3504af0.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@30bdfbd19f5a4aecb27546f3402363d2-00000000000a94b9-0005f56038be7272.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/user-1001@4eb77518f75346b098ac0ff6b46fa27b-00000000000b95bc-0005f777dacc893c.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@0005f78f96732787-752ac543038fc189.journal~  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@30bdfbd19f5a4aecb27546f3402363d2-00000000000b4d9a-0005f6eed50db92e.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/user-1000@07f0e6826aae45f19304eef8539ad6a2-00000000000aa1df-0005f560b9e56d4d.journal  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@0005f78fa3530d29-ee9522167f954620.journal~  
/var/log/journal/64a5d8763efc4f68a98665e5f30b596b/system@0005f78f939762e4-5f1fe564a7341afb.journal~
```

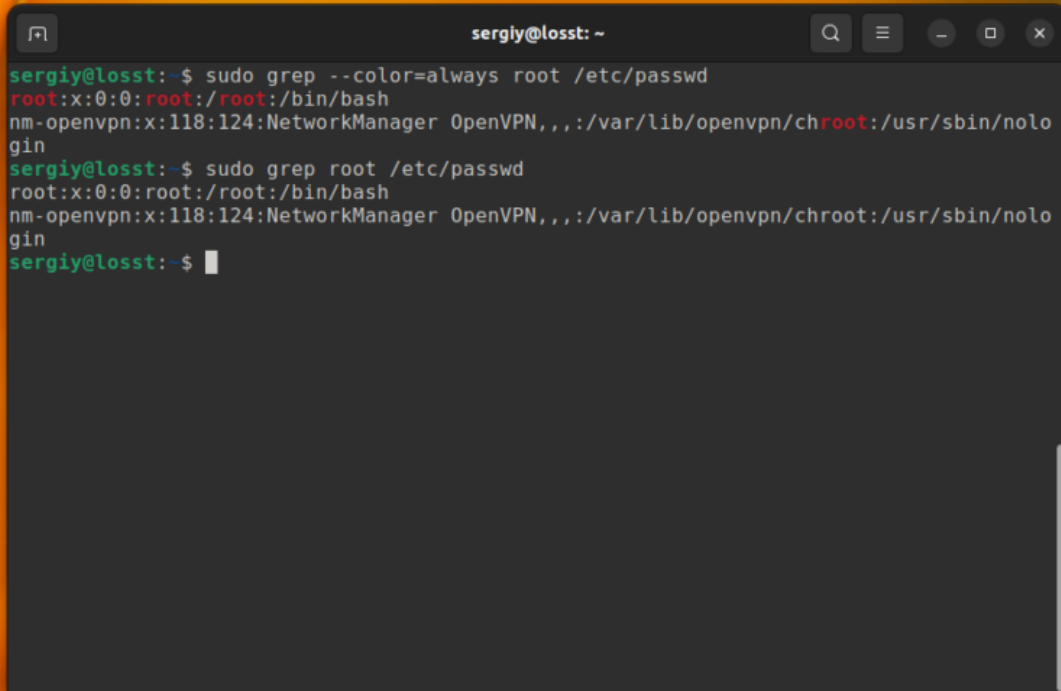
12. Цветной вывод

По умолчанию `grep` не будет подсвечивать совпадения цветом. Но в большинстве дистрибутивов прописан алиас для `grep`, который это включает. Однако, когда вы используете команду с `sudo` это работать не будет. Для включения подсветки вручную используйте опцию **`--color`** со значением **`always`**:

```
$ sudo grep --color=always root /etc/passwd
```

Получится:





```
sergiy@losst: ~  
sergiy@losst:~$ sudo grep --color=always root /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin  
sergiy@losst:~$ sudo grep root /etc/passwd  
root:x:0:0:root:/root:/bin/bash  
nm-openvpn:x:118:124:NetworkManager OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin  
sergiy@losst:~$
```

Выводы

Вот и всё. Теперь вы знаете что представляет из себя команда `grep` Linux, а также как ею пользоваться для поиска файлов и фильтрации вывода команд. При правильном применении эта утилита станет мощным инструментом в ваших руках. Если у вас остались вопросы, пишите в комментариях!

Обнаружили ошибку в тексте? Сообщите мне об этом. Выделите текст с ошибкой и нажмите `Ctrl+Enter`.

Похожие записи



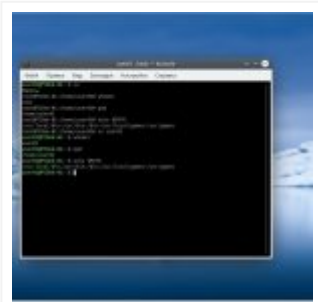
Команда dd Linux



Команда tar в Linux



Команда sed Linux



Команда su в Linux

Оцените статью

★★★★☆ (23 оценок, среднее: 4,17 из 5)



Статья распространяется под лицензией Creative Commons ShareAlike 4.0 при копировании материала ссылка на источник обязательна .

📁 [Команды](#)

Об авторе



ADMIN

Основатель и администратор сайта losst.ru, увлекаюсь открытым программным обеспечением и операционной системой Linux. В качестве основной ОС сейчас использую Ubuntu. Кроме Linux, интересуюсь всем, что связано с информационными технологиями и современной наукой.

18 комментариев к “Команда grep в Linux”



Andrei Oleynick (@makroprophus)

6 сентября, 2016 в 2:11 пп

Поправьте Тайтл на странице – Grep а Не GErp

[Ответить](#)



admin

6 сентября, 2016 в 2:49 пп

Поправил, спасибо.

[Ответить](#)



Dmytro

3 ноября, 2019 в 11:08 дп

С тайтлом разобрались, а URL так и остался герпом))

<https://losst.pro/gerp-poisk-vnutri-fajlov-v-linux>

[Ответить](#)



admin

3 ноября, 2019 в 4:56 пп

Нельзя менять URL, пусть будет уже.

[Ответить](#)**wdtime**19 сентября, 2016 в 10:09 дп

А в ссылке на статью так и остался анахронизм "gerp". С уважением, wdttime.ru

[Ответить](#)**Олег**1 декабря, 2017 в 8:32 дп

Описание опций перепутано

–An – показать вхождение и n строк после него;

–Bn – показать вхождение и n строк до него;

[Ответить](#)**Dzmitrock**22 ноября, 2019 в 7:06 пп

Можно. Редирект

[Ответить](#)**Иван**9 февраля, 2018 в 7:29 дп

выведем все сообщения за ноябрь

Все сообщения за 10-е ноября

[Ответить](#)**Порфирий**20 июля, 2018 в 8:14 пп

Добрый вечерок, а вот чтобы был поиск в папке по файлам *.html?

[Ответить](#)**Карина**16 августа, 2018 в 3:24 пп

```
cat *.html | grep -i (искомое)
```

[Ответить](#)**And**29 августа, 2018 в 12:21 пп

```
grep HEAD *.html
```

[Ответить](#)**And**29 августа, 2018 в 12:28 пп

это ошибочное описание.

-An – показать вхождение и n строк до него;

-Bn – показать вхождение и n строк после него;

а в примерах использование дано правильное

```
"grep -A4 "EE" /var/log/xorg.0.log
```

Выведет строку с вхождением и 4 строчки после неё"

after (-A) – после

before (-B) – до

Privacy

[Ответить](#)**Anna**[21 августа, 2020 в 11:19 дп](#)

Помогите с поиском текста на определенной веб-странице

[Ответить](#)**Filin**[18 ноября, 2020 в 12:01 пп](#)

```
curl example.com | grep Example
```

[Ответить](#)**Alex**[23 ноября, 2020 в 10:22 пп](#)

Как вывести текущие состояние сетевой карты .

Моя текущая ситуация .

```
iwconfig "NETWORK_CARD" | egrep -i 'Mode:Managed|Mode:Monitor' | awk '{print $4}'
```

```
#print Mode:Monitor
```

Не могу понять как реализовать вывод , получается играть только с awk в одну сторону мода сетевой карты .

[Ответить](#)**Filin**[9 декабря, 2020 в 4:11 пп](#)


```
curl example.com | grep Example
```

[Ответить](#)**user**[5 марта, 2021 в 9:37 пп](#)

Доброго времени суток. Вопрос такого плана: есть файл test.txt в нем в столбик указаны серийные номера, есть файл serial.csv содержащий модель устройства и серийный номер.

Делаю запрос: `grep -n -f test.txt serial.csv > out.log`

В файл out.log сохраняются все найденные номера. И тут вопрос, как вывести отдельным файлом те номера из файла test.txt что не было в serial.csv ?

[Ответить](#)**ВладимирП**[20 января, 2022 в 2:21 пп](#)

Опция '-с' для grep считает не количество вхождений, а количество строк, содержащих искомую информацию!

[Ответить](#)

Оставьте комментарий

[Privacy](#)

Имя *

Email

☐ Я прочитал и принимаю политику конфиденциальности. Подробнее [Политика конфиденциальности](#) *

Комментировать

English

Русский

Поиск

ПОИСК ПО КОМАНДАМ

Начните вводить команду

Поиск



Начните изучать
Linux прямо сейчас!

Карта сайта



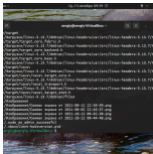
Как пользоваться
редактором Vim

Полезно

Лучшие

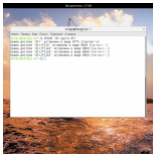
Свежие

Теги



Команда find в Linux

2021-10-17



Команда chmod Linux

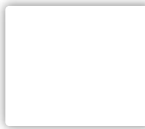
2020-04-13



Права доступа к файлам в Linux

2020-10-09

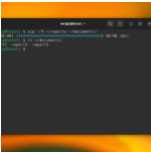
Настройка Cron



Privacy



2021-10-01



Копирование файлов в Linux

2023-03-03

РАССЫЛКА

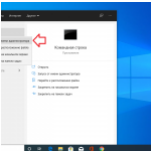
Ваш E-Mail адрес

☐ Я прочитал(а) и принимаю политику конфиденциальности

Sign up

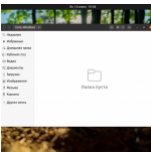
Windows

Списки



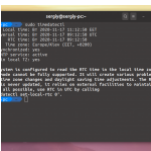
Восстановление Grub после установки Windows 10

2023-05-04



Ошибка Ubuntu не видит сеть Windows

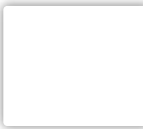
2023-02-19



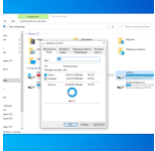
Сбивается время в Ubuntu и Windows

2023-02-19

Подключение ext4 в Windows



Privacy



2023-02-19

Смотреть ещё

ОБНАРУЖИЛИ ОШИБКУ?

Сообщите мне об этом. Выделите текст с ошибкой и нажмите Ctrl+Enter.

ВКОНТАКТЕ



>< komYounity

Мы создаём комьюнити, комьюнити создаёт Linux.

13 983 подписчика



Подписаться на новости

МЕТА

Регистрация
Войти
Лента записей
Лента комментариев

Privacy

СЛЕДИТЕ ЗА НАМИ В СОЦИАЛЬНЫХ СЕТЯХ

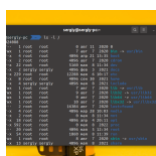


ИНТЕРЕСНОЕ



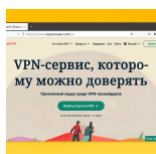
Что такое демоны в Linux

2021-06-01



Как правильно: папка или каталог в Linux

2022-05-10



Лучшие VPN сервисы для Linux

2022-10-10



Команды терминала Linux

2020-12-18

©Losst 2023 CC-BY-SA [Политика конфиденциальности](#)



Privacy